

4. MATLAB 프로그래밍

MATLAB Programming

Jin Hee Yoon

Dept. of Artificial Intelligence and Information Technology

Sejong University

Spring, 2026

- 2차 방정식의 근 구하기

예제 프로그램

```
disp('이차 방정식  $ax^2+bx+c=0$  의 근을 구하는 프로그램');
```

```
a = input('2차 항의 계수 (a)? ');
```

```
b = input('1차 항의 계수 (b)? ');
```

```
c = input('상수항의 계수 (c)? ');
```

```
r1 = (-b+sqrt(b^2-4*a*c))/2/a;
```

```
r2 = (-b-sqrt(b^2-4*a*c))/2/a;
```

```
fprintf('한 근은 %.2f 이고,\n다른 근은 %.2f 이  
다.\n', r1, r2);
```

- 2차 방정식의 근 구하기: 정상 사례

실행 결과

이차 방정식 $ax^2+bx+c=0$ 의 근을 구하는 프로그램

2차 항의 계수 (a)? 1

1차 항의 계수 (b)? 5

상수항의 계수 (c)? 4

한 근은 -1.00 이고, 다른 근은 -4.00 이다.

관계 연산자와 논리 연산자

- 2차 방정식의 근 구하기: 오류 사례

허근인 경우

이차 방정식 $ax^2+bx+c=0$ 의 근을 구하는 프로그램

2차 항의 계수 (a)? 1

1차 항의 계수 (b)? 0

상수항의 계수 (c)? 1

한 근은 0.00 이고, 다른 근은 -0.00 이다.

이차방정식이 아닌 경우

이차 방정식 $ax^2+bx+c=0$ 의 근을 구하는 프로그램

2차 항의 계수 (a)? 0

1차 항의 계수 (b)? 2

상수항의 계수 (c)? 3

한 근은 NaN 이고, 다른 근은 -Inf 이다.

- 방정식의 근 구하기

$$ax^2 + bx + c = 0$$

$$a \neq 0 \Rightarrow D = b^2 - 4ac$$

$$\begin{cases} D > 0 \Rightarrow \frac{-b \pm \sqrt{D}}{2a} \\ D = 0 \Rightarrow -\frac{b}{2a} \\ D < 0 \Rightarrow \frac{-b \pm j\sqrt{-D}}{2a} \end{cases}$$

$$a = 0 \Rightarrow \begin{cases} b \neq 0 \Rightarrow -\frac{c}{b} \\ b = 0, c = 0 \Rightarrow \text{부정(모든 실수)} \\ b = 0, c \neq 0 \Rightarrow \text{불능(해가 없음)} \end{cases}$$

- 방정식의 근 구하기: 순서도

1. 시작
2. a, b, c 값을 입력
3. $a = 0$ 여부를 판단
4. $a \neq 0$ 이면 $D = b^2 - 4ac$ 계산
5. $D > 0$, $D = 0$, $D < 0$ 에 따라 근 계산
6. $a = 0$ 이면 $b = 0$ 여부를 다시 판단
7. $b \neq 0$ 이면 $-c/b$ 계산
8. $b = 0$ 이면 $c = 0$ 여부에 따라 부정/불능 판단
9. 끝

- 방정식의 근 구하기: 순서도

fig_4_0.png

관계 연산자와 논리 연산자

- 관계 연산자
- 두 값 사이의 크기를 비교
- 참이면 1, 거짓이면 0 출력

종류	뜻
<	보다 작다
<=	보다 작거나 같다
>	보다 크다
>=	보다 크거나 같다
==	와 같다
~=	와 같지 않다

관계 연산자와 논리 연산자

- 관계 연산자 예

```
>> x=5;
>> y=1;

>> x<y
ans =      0
>> x<=y
ans =      0
>> x>y
ans =      1
>> x>=y
ans =      1
>> x==y
ans =      0
>> x~=y
ans =      1
```

- 관계연산자도 하나의 연산자로 값을 가짐

관계 연산자와 논리 연산자

- 관계 연산자: 벡터 및 행렬에도 적용 가능

```
>> x=1:5;
>> y=[-2 3 3 0 6];

>> x<y
ans =     0     1     0     0     1
>> x<=y
ans =     0     1     1     0     1
>> x>y
ans =     1     0     0     1     0
>> x>=y
ans =     1     0     1     1     0
>> x==y
ans =     0     0     1     0     0
>> x~=y
ans =     1     1     0     1     1
```

관계 연산자와 논리 연산자

- 논리 연산자
- 두 개 이상의 비교식(관계연산자를 사용한 식)을 결합할 때 사용

종류	뜻	종류	뜻
&	and	~	not
	or	xor	exclusive or

- $(x > y) \ \& \ (x > z)$: x 가 y, z 보다 모두 크면 참
- $(x > y) \ | \ (x > z)$: 둘 중 하나라도 만족하면 참
- $\sim(x)$: x 가 0이 아니면 거짓
- $\text{xor}(A, B)$: 둘 중 하나만 참일 때 참
- $\text{AND}(A, B)$, $\text{OR}(A, B)$, $\text{NOT}(A)$ 도 가능

- 논리 연산자 예

```
>> x=5;
>> y=3;
>> z=2;

>> (x<y) & (z>y)
ans =      0
>> (x>y) | (z>y)
ans =      1
>> ~(x>y)
ans =      0
>> not(x>y)
ans =      0
>> xor(x>y, z>y)
ans =      1
```

- 논리 연산자의 해석

$$x < a \text{ 또는 } x > b \implies (x < a) \mid (x > b)$$

$$a < x < b \implies (x > a) \& (x < b)$$

$$a = b = c \implies (a == b) \& (b == c)$$

- 관계연산자가 논리연산자보다 우선순위 높아서 괄호 없어도 실행되나 혼동이 있을 수 있으니 괄호를 넣는 것이 좋음.

관계 연산자와 논리 연산자

- 거짓은 0, 참은 1로 정의
- 좀 더 정확하게는 거짓은 0, 참은 0이 아닌 값으로 정의

```
>> x=5;
```

```
>> y=3;
```

```
>> (x>y) & 2
```

```
ans =
```

```
1
```

```
>> ~(5)
```

```
ans =
```

```
0
```

- 비교 연산자는 벡터와 스칼라 간의 연산이 가능함

```
>> x=1:5
```

```
x =
```

```
    1    2    3    4    5
```

```
>> x>3
```

```
ans =
```

```
    0    0    0    1    1
```

- 슈도코드(pseudocode)

1. 문제를 풀기 위해 필요한 단계들과 각 단계에서 해야 할 내용을 간추려 적음
2. 각 단계의 내용을 M-파일의 설명문(comment, 주석문)으로 만듦
3. 작성된 설명문 다음에 여기에 적합한 매트랩 명령어를 써넣음

- **의사코드 예:** mph를 ft/s로 환산하는 속도 변환 프로그램

```
% 환산하고 싶은 mph 값을 벡터로 만듦
% mph 값을 ft/s로 환산함
% mph 벡터와 ft/s 벡터를 결합하여 하나의 행렬로 만듦
% 표 제목을 만듦
% 표 항목의 이름을 만듦
% 표를 화면에 출력함
```

- **의사코드 예:** mph를 ft/s로 환산하는 속도 변환 프로그램

```
mph = 0:10:100; % 환산하고 싶은 mph 값을 벡터로 만듦

fps = mph*5280/3600; % mph 값을 ft/s로 환산함

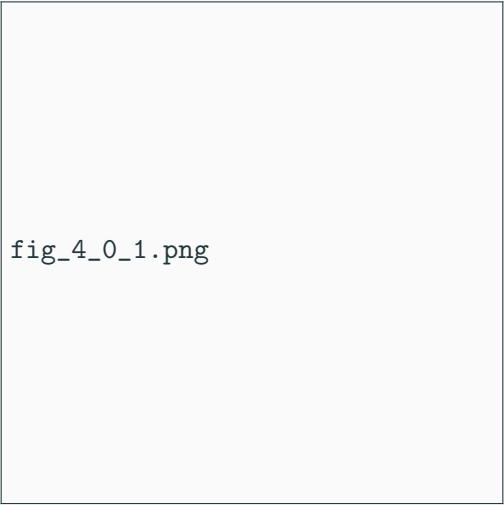
table = [mph;fps]; % mph 벡터와 ft/s 벡터를 결합하여 하나의 행렬로 만듦

disp('속도 변환 표') % 표 제목을 만듦

disp(' mph    ft/s') % 표 항목의 이름을 만듦

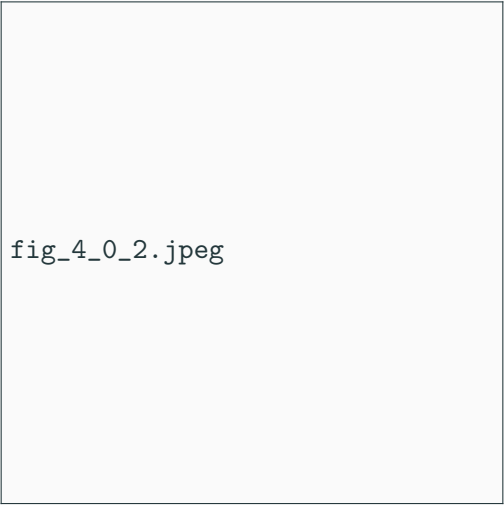
fprintf('%8.0f %8.2f \n', table) % 표를 화면에 출력함
```

- 순서도(flow chart)



fig_4_0_1.png

- 순서도(flow chart)



fig_4_0_2.jpeg

논리 함수

- `find(A)` : A 행렬에서 참(0이 아닌)인 원소의 위치를 찾는 함수

```
>> x=[1 0 -2];  
>> y=[0 0 9];  
  
>> find(x)  
ans =     1     3  
>> find(x & y)  
ans =     3  
>> A=[0 3 1;2 0 0]  
A =     0     3     1  
     2     0     0  
>> find(A)  
ans =     2  
     3  
     5
```

- **find 함수의 활용**
- find 함수를 이용하여 특정 조건에 맞는 원소의 위치와 원소를 찾을 수 있음

```
>> height = [63, 67, 65, 72, 69, 78, 75];
```

```
>> accept = find(height>=66)
```

```
accept =
```

```
     2     4     5     6     7
```

```
>> height(accept)
```

```
ans =
```

```
    67    72    69    78    75
```

- find 함수의 활용
- 주의: height>=66 은 행렬과 스칼라 간의 연산

```
>> height>=66
```

```
ans =
```

```
    0     1     0     1     1     1     1
```

- find 함수의 활용
- 키가 66인치 이상이면서 나이가 18살 이상 35세 미만인 경우

키(인치)	나이(살)
63	18
67	19
65	18
72	20
69	36
78	34
75	12

- find 함수의 활용
- 키와 나이 데이터를 행렬로 입력

```
>> appl = [63,18; 67,19; 65,18; 72,20; 69,36; 78,34; 75,12]
```

```
appl =
```

63	18
67	19
65	18
72	20
69	36
78	34
75	12

- find 함수의 활용
- 조건을 만족하는 행 번호 찾기

```
>> pass = find(app1(:,1)>=66 & app1(:,2)>=18 & app1(:,2)<35)
```

```
pass =
```

```
2
```

```
4
```

```
6
```

- **find 함수의 활용**
- 조건:
 - $A: \text{app1}(:,1) \geq 66$
 - $B: \text{app1}(:,2) \geq 18$
 - $C: \text{app1}(:,2) < 35$

키	나이	A	B	C	A&B&C
63	18	0	1	1	0
67	19	1	1	1	1
65	18	0	1	1	0
72	20	1	1	1	1
69	36	1	1	0	0
78	34	1	1	1	1
75	12	1	0	1	0

- **find** 함수의 활용
- 입력과 조건 검색을 한 번에 수행

```
>> appl = [63,18; 67,19; 65,18; 72,20; 69,36; 78,34; 75,12];  
>> pass = find(appl(:,1)>=66 & appl(:,2)>=18 & appl(:,2)<35)
```

```
pass =
```

```
2
```

```
4
```

```
6
```

- find 함수의 활용
- 조건을 만족하는 수험번호, 키, 나이를 함께 출력

```
>> pass = find(appl(:,1)>=66 & appl(:,2)>=18 & appl(:,2)<35);  
>> results = [pass, appl(pass,1), appl(pass,2)];  
>> fprintf('No. %4.0f: height = %4.0f, age = %4.0f', results')
```

```
No.    2: height =    67, age =    19  
No.    4: height =    72, age =    20  
No.    6: height =    78, age =    34
```

- **find 함수의 활용**
- 주의: `results`는 조건을 만족하는 행의 정보를 모은 행렬

```
>> results
```

```
results =
```

2	67	19
4	72	20
6	78	34

예제: 논리 함수

- `[row, col] = find(A)`
- A행렬에서 참인(0이 아닌) 원소의 위치를 찾아 행 번호는 `row`, 열 번호는 `col`

```
>> temp = [95.3 100.2 98.6;  
           97.4  99.2 98.9;  
           100.1 99.3 97.0];  
>> element = find(temp>99)  
element =  
     2  
     4  
     5  
     6  
>> [row, col] = find(temp>99)  
row =  
     1  
     2  
     2  
     3  
col =  
     2  
     1  
     2  
     2
```

예제: 논리 함수

$$x = \begin{bmatrix} 1 & 10 & 42 & 6 \\ 5 & 8 & 78 & 23 \\ 56 & 45 & 9 & 13 \\ 23 & 22 & 8 & 9 \end{bmatrix}, \quad y = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 10 & 12 \\ 7 & 21 & 27 \end{bmatrix}, \quad z = [10 \ 22 \ 5 \ 13]$$

- 각 행렬에서 값이 10보다 큰 원소의 **인덱스 번호**를 구하여라.
- 각 행렬에서 값이 10보다 큰 원소의 **행 번호와 열 번호**를 구하여라.
- 각 행렬에서 값이 10보다 큰 원소의 **실제 값**을 구하여라.
- 각 행렬에서 값이 10보다 큰 원소의 **개수**를 구하여라.

코드

- `idx_x = find(x>10)`
- `[row, col] = find(x>10)`
- `x(idx_x)`
- `length(idx_x)`

실습예제: 관계연산자, 논리연산자, find

1. $x = 7, y = 4$ 일 때, 다음의 결과를 구하여라. $x < y$, $x \geq y$, $x == y$, $x \neq y$
2. $x = 5, y = 3, z = 8$ 일 때, 다음 논리식의 결과를 구하여라.
 $(x > y) \ \& \ (z > x)$, $(x < y) \ | \ (z < x)$, $\sim(x == y)$
3. 벡터 $a = [2 \ 5 \ 8 \ 1 \ 9]$ 에 대하여 $a > 4$, $a \leq 5$ 의 결과를 구하여라.
4. 벡터 $score = [55 \ 72 \ 68 \ 81 \ 49 \ 90]$ 에서 70 이상인 원소의 **인덱스 번호**와 **실제 값**을 구하여라.
5. 행렬

$$A = \begin{bmatrix} 3 & 12 & 5 \\ 18 & 7 & 21 \\ 4 & 25 & 9 \end{bmatrix}$$

에 대하여, 10보다 큰 원소의 **인덱스 번호**, **행 번호**와 **열 번호**, **실제 값**을 구하여라.

실습예제 코드답 1

```
>> x = 7; y = 4;
```

```
>> x<y
```

```
ans =
```

```
0
```

```
>> x>=y
```

```
ans =
```

```
1
```

```
>> x==y
```

```
ans =
```

```
0
```

```
>> x~=y
```

```
ans =
```

```
1
```

실습예제 코드답 2

```
>> x = 5; y = 3; z = 8;
```

```
>> (x>y) & (z>x)
```

```
ans =
```

```
1
```

```
>> (x<y) | (z<x)
```

```
ans =
```

```
0
```

```
>> ~(x==y)
```

```
ans =
```

```
1
```

실습예제 코드답 3

```
>> a = [2 5 8 1 9];
```

```
>> a>4
```

```
ans =
```

```
    0    1    1    0    1
```

```
>> a<=5
```

```
ans =
```

```
    1    1    0    1    0
```

실습예제 코드답 4

```
>> score = [55 72 68 81 49 90];  
  
>> idx = find(score>=70)  
idx =  
     2     4     6  
  
>> score(idx)  
ans =  
    72    81    90
```

실습예제 코드답 5

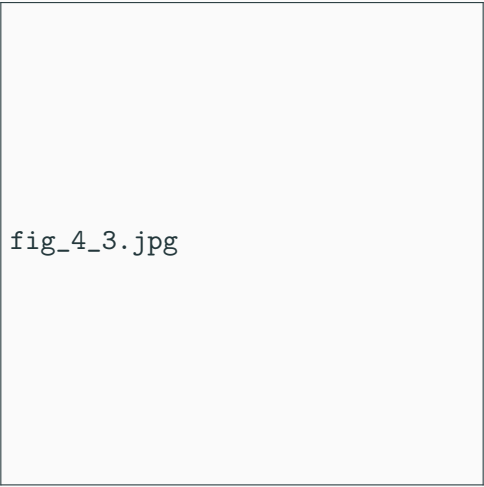
```
>> A = [3 12 5; 18 7 21; 4 25 9];  
>> idxA = find(A>10)  
idxA =  
     2  
     4  
     6  
     8  
  
>> [row, col] = find(A>10)  
row =  
     2  
     1  
     3  
     2  
  
col =  
     1  
     2  
     2  
     3  
  
>> A(idxA)  
ans =  
    18  
    12  
    25  
    21
```

- if 문은 논리식이 참일 때 실행문을 수행한다.
- 모든 if 문은 반드시 end 로 끝나야 한다.

```
if logical_expression
    statements
end
```


- else 문은 논리식이 거짓일 때 수행할 실행문 그룹을 제공한다.

```
if logical_expression
    statements_group_1
else
    statements_group_2
end
```



fig_4_3.jpg

- 논리식은 배열도 이용 가능.
- 배열 논리식의 테스트 결과는 모든 원소가 참일 경우에만 참이 되는 방식.
- 일부 원소만 조건을 만족하는 경우 예상과 다르게 수행 가증
- 실행 의도는 “some elements are negative” 와 같은 메시지를 출력하는 것이다.

else 구조에서의 논리식-2

```
x = [4,-9,25];  
if x < 0  
    disp('some elements are negative')  
else  
    y = sqrt(x)  
end  
=====
```

```
x = [4,-9,25];  
if any(x < 0)  
    disp('some elements are negative')  
else  
    y = sqrt(x)  
end
```

else 구조에서의 논리식-3

- $x \geq 0$ 도 벡터 논리식을 반환함.
- $x \geq 0$ 가 $[1,0,1]$ 이므로 전체 테스트는 거짓임.
- 참고) 출력 메시지는 코드 안에 한글 대신 영문 문자열을 사용하는 것이 안전함.

```
x = [4,-9,25];  
if x >= 0  
    y = sqrt(x)  
else  
    disp('some elements are negative')  
end
```

이중 if 문

- 중첩된 if 문은 하나의 논리식으로 합칠 수 있음.

```
if logical_expression_1
    if logical_expression_2
        statements
    end
end
```

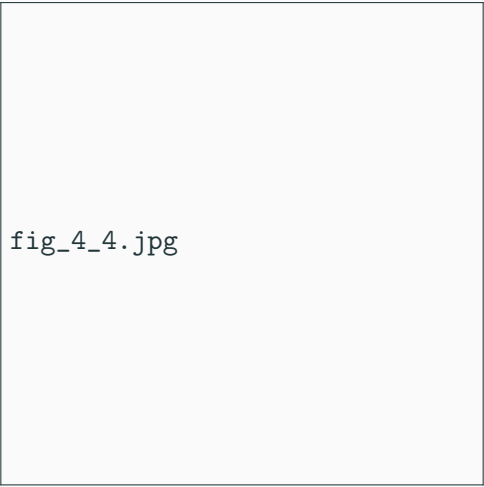
```
if logical_expression_1 & logical_expression_2
    statements
end
```

elseif 문

- elseif 문은 여러 개의 조건을 순차적으로 검사한다.
- else 는 맨 마지막에 위치해야 한다.

```
if logical_expression_1
    statements_group_1
elseif logical_expression_2
    statements_group_2
else
    statements_group_3
end
```

일반적인 if-elseif-else 구조의 흐름도



fig_4_4.jpg

if-elseif-else 구조의 예

- 다음은 입력값의 범위에 따라 서로 다른 수식을 선택하는 예이다.

```
if x > 10
    y = log(x)
elseif x >= 0
    y = sqrt(x)
else
    y = exp(x) - 1
end
```

단일 if 구조 예제: 홀수/짝수 판별

- 사용자가 입력한 정수가 짝수인지 홀수인지 판별하는 예이다.
- 나머지 연산 `rem(n,2)`를 이용한다.

MATLAB 코드

```
n = input('정수를 입력하세요: ');

if rem(n,2) == 0
    fprintf('%d 는 짝수입니다.\n', n);
end

if rem(n,2) ~= 0
    fprintf('%d 는 홀수입니다.\n', n);
end
```

if/else 구조 예제: 홀수/짝수 판별

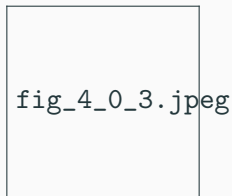
- 같은 문제를 if/else로 더 간단히 작성할 수 있다.

MATLAB 코드

```
n = input('정수를 입력하세요: ');

if rem(n,2) == 0
    fprintf('%d 는 짝수입니다.\n', n);
else
    fprintf('%d 는 홀수입니다.\n', n);
end
```

if/else 구조 응용예제: 급수탑 물의 부피구하기



- 부표의 위치 h 에 따라 물의 체적 계산식이 달라진다.
- $h \leq 19$ 이면 아래쪽 원기둥 부분만 고려한다.
- $h > 19$ 이면 원기둥 부분과 위쪽 절두원뿔 부분을 함께 고려한다.

if/else 구조 응용: 급수탑 물의 체적

$$V = \pi(12.5)^2 h \quad (h \leq 19)$$

$$r_h = 12.5 + \frac{10.5}{14}(h - 19)$$

$$V = \pi(12.5)^2 \cdot 19 + \frac{1}{3}\pi(h - 19)(12.5^2 + 12.5r_h + r_h^2) \quad (h > 19)$$

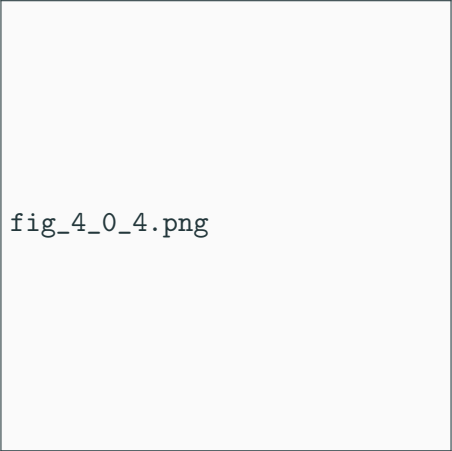
급수탑 물의 체적 계산 함수

MATLAB 함수

```
function v = watervol(h)
% h : 물의 높이(m)
% v : 물의 체적(m^3)

if h <= 19
    v = pi*12.5^2*h;
else
    rh = 12.5 + 10.5*(h-19)/14;
    v = pi*12.5^2*19 + pi*(h-19)*(12.5^2 + 12.5*rh + rh^2)/3;
end
```

if – elseif – else 예제: 운전면허 구분



fig_4_0_4.png

if – elseif – else 예제: 운전면허 구분

MATLAB 코드

```
if age < 16
    disp('아직은 운전면허증을 발급받을 수 있는 나이
가 아닙니다.')
```

```
elseif age < 18
    disp('청소년 운전면허증을 발급받을 수 있습니
다.')
```

```
elseif age < 70
    disp('일반 운전면허증을 발급받을 수 있습니다.')
```

```
else
    disp('70세 이상은 특수 운전면허증을 발급받아
야 합니다.')
```

```
end
```


예제: 학점 부여 프로그램

- 점수에 따라 학점을 부여하는 문제를 생각할 수 있다.

학점	점수 구간
A	90-100
B	80-89
C	70-79
D	60-69
F	60 미만

- 이러한 경우 **if – elseif – else** 구조가 적합하다.

학점 부여 함수 1

MATLAB 함수

```
function results = grade(x)

if x >= 90
    results = 'A';
elseif x >= 80
    results = 'B';
elseif x >= 70
    results = 'C';
elseif x >= 60
    results = 'D';
else
    results = 'F';
end
```

학점 부여 함수 1의 주의점

- 입력은 점수이고, 출력은 문자(character)이다.
- 위 함수는 점수 범위를 따로 검사하지 않는다.
- 따라서 100점 초과 또는 0점 미만인 값에도 학점이 부여될 수 있다.
- 실전에서는 **입력값의 유효성 검사**가 필요하다.

학점 부여 함수 2: 입력값 검사 포함

MATLAB 함수

```
function results = grade(x)

if x >= 0 && x <= 100
    if x >= 90
        results = 'A';
    elseif x >= 80
        results = 'B';
    elseif x >= 70
        results = 'C';
    elseif x >= 60
        results = 'D';
    else
        results = 'F';
    end
else
    results = '잘못된 입력입니다.';
end
```

& 와 &&의 차이

구분	&	&&
의미	element-wise AND	short-circuit AND
사용 대상	벡터, 행렬, 스칼라	스칼라 논리값
동작 방식	양쪽을 모두 계산한 뒤 원소별로 AND 수행	앞 조건이 거짓이면 뒤 조건은 계산하지 않음
결과 형태	논리 배열 또는 스칼라	하나의 논리값
주요 사용 위치	배열 조건 비교, 논리 인덱싱	if, while의 스칼라 조건식
예시	(x>0) & (x<10)	a>0 && a<10
주의점	벡터 결과가 나올 수 있으므로 if에서 직접 쓰면 주의 필요	배열에는 사용할 수 없음

- &는 원소별 비교에 사용한다.
- &&는 하나의 조건이 참인지 거짓인지 판단할 때 사용한다.

방정식의 근 구하기

- 이차방정식

$$ax^2 + bx + c = 0$$

의 해는 계수의 값에 따라 경우가 달라진다.

- $a \neq 0$ 이면 판별식

$$D = b^2 - 4ac$$

을 이용하여 해의 형태를 결정한다.

- $a = 0$ 이면 일차방정식 또는 특수한 형태가 된다.

$$D > 0 \Rightarrow \text{서로 다른 두 실근}$$

$$D = 0 \Rightarrow \text{중근}$$

$$D < 0 \Rightarrow \text{두 허근}$$

방정식의 근을 구하는 함수 root2

MATLAB 함수

```
function x = root2(a,b,c)
if a ~= 0
    D = b^2 - 4*a*c;
    if D == 0
        x = -b/(2*a);
    elseif D > 0
        x(1) = (-b + sqrt(D))/(2*a);
        x(2) = (-b - sqrt(D))/(2*a);
    else
        x(1) = (-b + 1i*sqrt(-D))/(2*a);
        x(2) = (-b - 1i*sqrt(-D))/(2*a);
    end
end
```

뒷장 계속

방정식의 근을 구하는 함수 root2

MATLAB 함수

```
else
    if b ~= 0
        x = -c/b;
    else
        if c == 0
            disp('부정 : 모든 실수가 근');
        else
            disp('불능 : 해가 없음');
        end
    end
end
end
```


switch 선택구조문

- switch 는 입력 식을 기준으로 여러 경우를 비교할 때 사용한다.
- otherwise 는 어떤 경우에도 해당하지 않을 때 실행되는 부분이다.

```
switch input_expression
  case value1
    statements_group_1
  case value2
    statements_group_2
  otherwise
    statements_group_n
end
```

예제: switch 문 (각도를 이용한 나침반 방향 예제)

```
x = input('0부터 359까지의 숫자를 이용하여 각도를 입력하세요: ');
switch x
    case 0
        disp('east')
    case 45
        disp('northeast')
    case 90
        disp('north')
    case 135
        disp('northwest')
    case 180
        disp('west')
    case 225
        disp('southwest')
    case 270
        disp('south')
    case 315
        disp('southeast')
    otherwise
        disp('unknown direction')
end
```

실습문제: 논리제어문(선택구조문)

1. x 가 0보다 크면 $y = x^2$ 을 계산하고, 그렇지 않으면 $y = 0$ 을 출력하는 if 문을 작성하시오.
2. 벡터 x 의 원소 중에서 음수만 찾아 출력하는 코드를 작성하시오.
3. 점수 $score$ 에 따라 A, B, C를 출력하는 if-elseif-else 문을 작성하시오.

실습문제: 논리제어문(선택구조문)

4. 다음 벡터 x 에 대하여 음수 원소가 하나라도 있으면 그 개수를 구하고, 음수 원소가 없으면 'no negative values'를 출력하는 코드를 작성하시오.

$$x = [3, -2, 5, -7, 0, -1]$$

실습문제: 논리제어문(선택구조문)

5. 다음 벡터 x 의 각 원소에 대하여

- 양수이면 제곱근을 취하고
- 0이면 그대로 두고
- 음수이면 절댓값으로 바꾸는

새 벡터 y 를 만드는 코드를 작성하시오. 단, `for` 문과 `if-elseif-else` 문을 함께 사용하시오.

$$x = [9, -4, 0, 16, -25, 1]$$

6. 다음 문자열 변수 `cmd`의 값에 따라 서로 다른 메시지를 출력하는 `switch` 문을 작성하시오.

- 'start' 이면 'system on'을 출력하시오.
- 'stop' 이면 'system off'를 출력하시오.
- 'pause' 이면 'system paused'를 출력하시오.
- 그 외의 경우에는 'invalid command'를 출력하시오.

실습문제 1 코드답

```
x = 5;  
  
if x > 0  
    y = x^2;  
else  
    y = 0;  
end
```

실습문제 2 코드답

```
x = [3, -1, 5, -7, 2];
```

```
negative = x(x < 0)
```


실습문제 3 코드답

```
score = 85;

if score >= 90
    grade = 'A';
elseif score >= 80
    grade = 'B';
else
    grade = 'C';
end
```

실습문제 4 코드답

```
x = [3, -2, 5, -7, 0, -1];  
  
neg_idx = (x < 0);  
  
if any(neg_idx)  
    neg_count = length(x(neg_idx))  
else  
    disp('no negative values')  
end
```

실습문제 5 코드답

```
x = [9, -4, 0, 16, -25, 1];  
y = zeros(size(x));  
  
for k = 1:length(x)  
    if x(k) > 0  
        y(k) = sqrt(x(k));  
    elseif x(k) == 0  
        y(k) = x(k);  
    else  
        y(k) = abs(x(k));  
    end  
end  
  
y
```

실습문제 6 코드답

```
cmd = input('start, stop, pause 중 하나를 입력하세요: ', 's');  
switch cmd  
    case 'start'  
        disp('system on')  
    case 'stop'  
        disp('system off')  
    case 'pause'  
        disp('system paused')  
    otherwise  
        disp('invalid command')  
end
```

반복 구조문 (Loop)

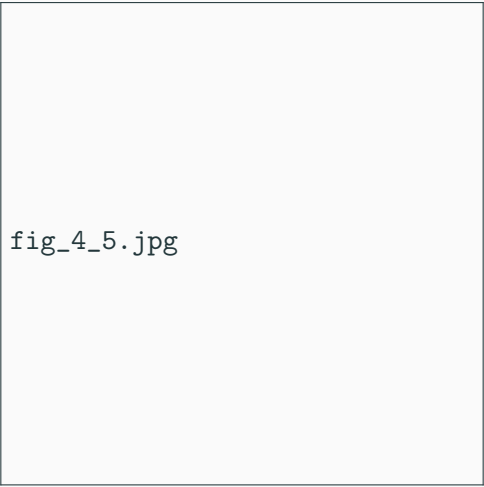
- 반복 구조는 명령어 그룹을 여러 번 수행하기 위해 사용된다.
- 반복 수행 시 변수 값이 매번 갱신된다.
- 대표적인 반복 구조
 - **for-end loop** : 반복 횟수가 정해진 경우
 - **while-end loop** : 조건이 만족되는 동안 반복

for-end loop 기본 구조

- 반복 횟수가 정해진 경우 사용하는 구조이다.

```
for 인덱스 = 배열 (벡터 또는 콜론연산자 벡터)
    수행할 명령어들
end
```

- 인덱스 값이 순차적으로 변하면서 반복 실행된다.



fig_4_5.jpg

for-end loop 예제

```
for k = [1 4 7 10]
    k^3
end
```

- k 값이 $1 \rightarrow 4 \rightarrow 7 \rightarrow 10$ 순서로 변하면서 실행된다.

예제: 일정 간격 수의 제공

```
for k = 1:3:10  
    k^2  
end
```

```
[1 4 7 10].^2
```

```
(1:3:10).^2
```

예제: 1부터 N까지의 합

for 문

```
N = input('N= ');  
sumN = 0;  
  
for k = 1:N  
    sumN = sumN + k;  
end
```

벡터화

```
sumN = sum(1:N);
```

예제: 홀수의 합

for 문

```
for k = 1:N
    if rem(k,2) == 1
        sumN = sumN + k;
    end
end
```

벡터화

```
sumN = sum(1:2:N);
```

예제: 완전제곱수의 합

확인 후 더하기

```
for k = 1:N
    if rem(sqrt(k),1) == 0
        sumN = sumN + k;
    end
end
```

완전제곱수 생성 후 더하기

```
for k = 1:sqrt(N)
    sumN = sumN + k^2;
end
```

벡터 만들기

for 문

```
for k = 1:N  
    a(k) = k^3;  
end
```

벡터화

```
a = (1:N).^3;
```

열 벡터 만들기

for 문

```
for k = 1:N  
    a(k,1) = k^3;  
end
```

벡터화

```
a = ((1:N).^3)';
```

행렬 만들기 (기본)

열 단위 할당

```
for k = 1:N
    a(:,k) = k^3 * ones(N,1);
end
```

행 단위 할당

```
N = input('N= ');
for k = 1:N
    a(k,:) = (1:N).^3;
end
```

행렬 만들기 (응용)

이중 for 문

```
for i = 1:N
    for j = 1:N
        a(i,j) = j^3;
    end
end
```

```
for i = 1:N
    for j = 1:N
        a(i,j) = (i+j-1)^3;
    end
end
```


행렬 만들기 (조건 포함)

```
for i = 1:N
    for j = 1:N
        if j >= i
            a(i,j) = 1;
        else
            a(i,j) = 0;
        end
    end
end
```

예제: 특정 조건 개수 세기

for 문

```
scores = [76,45,98,97];  
count = 0;  
  
for k = 1:length(scores)  
    if scores(k) > 90  
        count = count + 1;  
    end  
end  
  
disp(count)
```

find 함수

```
idx = find(scores > 90);  
length(idx)
```

예제: 도(degree)-라디안(radian) 변환 테이블

- 10° 부터 360° 까지 라디안 값으로 변환한다.

```
clear, clc;

for k = 1:36
    degree(k) = k*10;
    radian(k) = degree(k)*pi/180;
end

table = [degree; radian];

disp('Degree-to-Radian Table')
disp(' degree      radian ')
fprintf('%8.0f %8.2f \n', table)
```

예제: 벡터화를 이용한 도-라디안 변환

- 반복문 없이 한 번에 계산 가능

```
clear, clc

degree = 10:10:360;
radian = degree*pi/180;

table = [degree; radian];

disp('Degree-to-Radian Table')
disp(' degree      radian ')
fprintf('%8.0f %8.2f \n', table)
```

예제: for문을 이용한 도-라디안 변환 출력

```
clear, clc

for k = 1:36
    degree(k) = k*10;
    radian(k) = degree(k)*pi/180;
end

disp('Degree-to-Radian Table')
disp(' degree      radian ')

for k = 1:36
    fprintf('%8.0f %8.2f \n', degree(k), radian(k))
end
```

예제: 도-라디안 변환 출력 방식 비교

- 아래와 같이 행렬 전체를 fprintf에 넘겨 한 번에 출력할 수도 있다.
- MATLAB은 열 우선(column-wise) 방식으로 데이터를 읽기 때문에 출력 순서를 주의해야 한다.

비교 코드

```
table = [degree; radian];  
  
fprintf('%8.0f %8.2f \n', table)
```

예제: 삼각함수 표 만들기

```
clear, clc

for k = 1:90
    tbl_sin(k) = sin(k*pi/180);
    tbl_cos(k) = cos(k*pi/180);
    tbl_tan(k) = tan(k*pi/180);
end

disp('Trigonometric Table')
disp(' deg      Sine      Cosine      Tangent')

for k = 1:90
    fprintf('%3d %8.4f %8.4f %8.4f \n', ...
            k, tbl_sin(k), tbl_cos(k), tbl_tan(k))
end
```

예제: factorial 함수

```
function output = fact(x)

a = 1;

for k = 1:x
    a = a*k;
end

output = a;
```


예제: 입력 오류를 고려한 factorial 함수

```
function output = fact(x)

if (length(x) > 1) | (x < 0) | (rem(x,1) ~= 0)
    output = 'Input must be a positive integer.';
else
    a = 1;
    for k = 1:x
        a = a*k;
    end
    output = a;
end
```

예제: Taylor 급수를 이용한 sin 함수

```
function y = Tsin(x, N)

xr = x*pi/180;

y = 0;
for k = 0:N
    y = y + (-1)^k * xr^(2*k+1) / factorial(2*k+1);
end
```

while-end loop

- while문은 반복 횟수를 미리 알 수 없을 때 자주 사용한다.
- 조건식이 참인 동안 명령문을 계속 반복한다.

```
while condition
    statements
end
```

- 조건식이 거짓이 되면 반복이 종료된다.

예제: while-end loop 간단한 예

while 문

```
k = 0;  
  
while k < 3  
    k = k + 1  
end
```

실행 결과

```
k =  
    1  
  
k =  
    2  
  
k =  
    3
```

- 반복문 안에서 k 값이 바뀌므로 조건이 거짓이 되어 종료된다.

while문 작성 시 주의점

- 조건식에 사용된 변수 중 적어도 하나는 while 내부에서 값이 변경되어야 한다.
- 그렇지 않으면 **무한 루프**가 발생한다.

예제: 1부터 N까지의 합

while 문

```
N = input('N= ');
sumN = 0;
k = 1;

while k <= N
    sumN = sumN + k;
    k = k + 1;
end
```

for 문

```
N = input('N= ');
sumN = 0;

for k = 1:N
    sumN = sumN + k;
end
```

벡터화

```
N = input('N= ');
sumN = sum(1:N);
```

예제: 완전제곱수의 합

for 문

```
N = input('N= ');
sumN = 0;

for k = 1:N
    if rem(sqrt(k),1) == 0
        sumN = sumN + k;
    end
end
```

벡터화

```
N = input('N= ');
A = 1:N;

idx = find(rem(sqrt(A),1) == 0);
sumN = sum(A(idx));
```

예제: 완전제곱수의 합 (벡터화)

```
N = input('N= ');  
A = 1:N;  
  
idx = find(rem(sqrt(A),1) == 0);  
sumN = sum(A(idx));
```


예제: 완전제곱수 생성 후 합

while 문

```
N = input('N= ');
sumN = 0;
k = 1;

while k <= sqrt(N)
    sumN = sumN + k^2;
    k = k + 1;
end
```

for 문

```
N = input('N= ');
sumN = 0;

for k = 1:sqrt(N)
    sumN = sumN + k^2;
end
```

예제: 벡터 만들기

```
N = input('N= ');  
a = (1:N).^3;
```

예제: 행렬 만들기

```
N = input('N= ');

for i = 1:N
    for j = 1:N
        a(i,j) = (i+j-1)^3;
    end
end
```

예제: 90점 이상 개수 세기

```
scores = [76,45,98,97];  
count = 0;  
  
for k = 1:length(scores)  
    if scores(k) > 90  
        count = count + 1;  
    end  
end  
  
disp(count)
```

실습문제: for 문과 while문

1. 1부터 20까지의 합

for 문을 사용하여 $1 + 2 + \dots + 20$ 을 계산하시오.

2. 1부터 10까지의 짝수 출력

for 문을 사용하여 1부터 10까지의 짝수를 차례로 출력하시오.

3. 1부터 N까지의 제곱합

사용자로부터 N을 입력받아 $\sum_{k=1}^N k^2$ 를 계산하시오.

4. while 문으로 10부터 1까지 출력

while 문을 사용하여 10, 9, 8, ..., 1을 출력하시오.

5. 벡터 원소의 합

벡터 $x = [3 \ 5 \ 2 \ 7 \ 4]$ 의 원소 합을 for 문으로 구하시오.

6. 사용자로부터 정수 N 을 입력받아, N 보다 작거나 같은 **피보나치 수열**의 항들을 차례로 출력하시오.
단, while 문을 사용하시오.

7. 사용자로부터 정수 N 을 입력받아, 1부터 N 까지의 수 중에서 3의 배수이면서 5의 배수는 아닌 수들의 합을 구하시오.
단, for 문과 if 문을 함께 사용하시오.

8. 벡터

$$x = [4, -3, 0, 7, -1, 5]$$

의 각 원소에 대하여

- 양수이면 제공,
- 0이면 그대로 유지,
- 음수이면 절댓값

을 취하여 새로운 벡터 y 를 만드시오.

단, for 문과 if-elseif-else 문을 사용하시오.

실습문제: for 문과 while문

9. 사용자로부터 정수 N 을 입력받아 다음과 같은 $N \times N$ 행렬을 만드시오.

$$A = \begin{bmatrix} 1 & 2 & 3 & \cdots & N \\ 2 & 4 & 6 & \cdots & 2N \\ 3 & 6 & 9 & \cdots & 3N \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ N & 2N & 3N & \cdots & N^2 \end{bmatrix}$$

즉, $A(i, j) = ij$ 이다.

단, for 문의 중첩 반복을 사용하시오.

실습문제: for 문과 while문

10. 초기값 $x = 100$ 에서 시작하여 다음 규칙을 반복하시오.

- x 가 짝수이면 $x = x/2$
- x 가 홀수이면 $x = 3*x + 1$

x 가 1이 될 때까지 이 과정을 반복하고, 중간에 나오는 값들을 모두 벡터에 저장하시오.

단, while 문을 사용하시오.

실습문제 1 코드답안

```
sumVal = 0;

for k = 1:20
    sumVal = sumVal + k;
end

disp(sumVal)
```

실습문제 2 코드답안

```
for k = 2:2:10  
    disp(k)  
end
```

실습문제 3 코드답안

```
N = input('N= ');  
sumSq = 0;  
  
for k = 1:N  
    sumSq = sumSq + k^2;  
end  
  
disp(sumSq)
```

실습문제 4 코드답안

```
k = 10;

while k >= 1
    disp(k)
    k = k - 1;
end
```

실습문제 5 코드답안

```
x = [3 5 2 7 4];  
s = 0;  
  
for k = 1:length(x)  
    s = s + x(k);  
end  
  
disp(s)
```

실습문제 6 코드답안

```
N = input('N= ');

a = 1;
b = 1;

disp(a)
disp(b)

while true
    c = a + b;
    if c > N
        break
    end
    disp(c)
    a = b;
    b = c;
end
```


실습문제 7 코드답안

```
N = input('N= ');  
sumVal = 0;  
  
for k = 1:N  
    if rem(k,3) == 0 && rem(k,5) ~= 0  
        sumVal = sumVal + k;  
    end  
end  
  
disp(sumVal)
```

실습문제 8 코드답안

```
x = [4 -3 0 7 -1 5];  
y = zeros(size(x));  
  
for k = 1:length(x)  
    if x(k) > 0  
        y(k) = x(k)^2;  
    elseif x(k) == 0  
        y(k) = x(k);  
    else  
        y(k) = abs(x(k));  
    end  
end  
  
disp(y)
```

실습문제 9 코드답안

```
N = input('N= ');

for i = 1:N
    for j = 1:N
        A(i,j) = i*j;
    end
end

disp(A)
```

실습문제 10 코드답안

```
x = 100;
values = x;

while x ~= 1
    if rem(x,2) == 0
        x = x/2;
    else
        x = 3*x + 1;
    end
    values(end+1) = x;
end

disp(values)
```